# Lekcja

# Temat: Wstęp do algorytmów.

**Algorytm** to uporządkowany, jednoznaczny zestaw kroków prowadzących do rozwiązania konkretnego problemu lub wykonania zadania.

**Przykład (z życia codziennego)**

Algorytm robienia herbaty:

1. Zagotuj wodę
2. Włóż torebkę herbaty do kubka
3. Zalej wrzątkiem
4. Odczekaj 3–5 minut
5. Wyjmij torebkę

## Zalety algorytmów

1. **Precyzyjność** – jasno określone kroki minimalizują błędy.
2. **Powtarzalność** – zawsze dają ten sam wynik dla tych samych danych wejściowych.
3. **Automatyzacja** – umożliwiają działanie programów komputerowych.
4. **Oszczędność czasu** – dobrze zaprojektowany algorytm przyspiesza pracę.
5. **Rozwiązywanie złożonych problemów** – dzielą duże problemy na mniejsze kroki.

## Wady algorytmów

1. **Brak elastyczności** – działają tylko według zaprogramowanych reguł.
2. **Mogą być nieefektywne** – źle zaprojektowany algorytm działa wolno.
3. **Złożoność** – trudne algorytmy są skomplikowane do zrozumienia i utrzymania.
4. **Wymagają dokładnych danych wejściowych** – błędne dane = błędny wynik.
5. **Koszt projektowania** – stworzenie dobrego algorytmu może być czasochłonne.

# Lekcja

# Temat: Podstawowe algorytmy

## Dodać implementacje w C++

### 1. Sortowanie bąbelkowe (Bubble Sort)

**Wyjaśnienie:** To prosty algorytm sortowania, który wielokrotnie przechodzi przez listę, porównując sąsiednie elementy i zamieniając je miejscami, jeśli są w złej kolejności. Proces powtarza się, aż lista będzie posortowana. Jest nieefektywny dla dużych zbiorów danych (złożoność $O(n^2)$), ale łatwy do zrozumienia. Nazwa pochodzi od "bąbelków" – większe elementy "wypływają" na koniec listy.

**Przykład kodu:**

```python
def bubble_sort(arr):
    n = len(arr)  #
    for i in range(n):  # Pętla zewnętrzna: dla każdej iteracji
        swapped = False  # Flaga do optymalizacji (czy była zamiana)
        for j in range(0, n - i - 1):  # Pętla wewnętrzna: porównaj sąsiednie elementy
            if arr[j] > arr[j + 1]:  # Jeśli bieżący > następny, zamień
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr


lista = [64, 34, 25, 12, 22, 11, 90]
posortowana = bubble_sort(lista)
print(posortowana)
```

### 2. Wyszukiwanie binarne (Binary Search)

**Wyjaśnienie:** Algorytm wyszukuje element w posortowanej liście, dzieląc ją na pół za każdym razem. Porównuje szukany element z środkowym elementem listy i eliminuje połowę, w której elementu nie ma. Jest bardzo efektywny (złożoność $O(\log n)$), ale wymaga, aby lista była posortowana. To jak szukanie słowa w słowniku – otwierasz na środku i sprawdzasz, czy iść w lewo czy w prawo.

**Przykład kodu:**

```python
def binary_search(arr, target, low=0, high=None):
    if high is None:
        high = len(arr) - 1  # Inicjalizacja high na końcu listy
    if low > high:
        return -1  # Nie znaleziono

    mid = (low + high) // 2  # Środkowy indeks
    if arr[mid] == target:
        return mid  # Znaleziono
    elif arr[mid] < target:
        return binary_search(arr, target, mid + 1, high)  # Prawa połowa
    else:
        return binary_search(arr, target, low, mid - 1)  # Lewa połowa


posortowana_lista = [11, 12, 22, 25, 34, 64, 90]
indeks = binary_search(posortowana_lista, 25)
print(indeks)  # Wyjście: 3 (indeks elementu 25)
```

Albo wersja iteracyjna (bez rekurencji, lepsza dla dużych list, bo unika przepełnienia stosu):

```python
def binary_search_iterative(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1  # Nie znaleziono

# Przykład:
posortowana_lista = [11, 12, 22, 25, 34, 64, 90]
indeks = binary_search_iterative(posortowana_lista, 64)
print(indeks)  # Wyjście: 5
```

## 3. Obliczanie silni (rekurencyjnie)

Silnia (oznaczana jako n!) to iloczyn wszystkich dodatnich liczb całkowitych od 1 do n. Na przykład, 5! = 5 × 4 × 3 × 2 × 1 = 120. Algorytm rekurencyjny oblicza silnię

poprzez wywoływanie samej siebie z mniejszym argumentem, aż dojdzie do przypadku bazowego.

```
def silnia(n):
    if n <= 1:  # Przypadek bazowy
        return 1
    else:
        return n * silnia(n - 1)  # Krok rekurencyjny

# Przykład użycia:
wynik = silnia(5)
print(wynik)  # Wyjście: 120

# Inne przykłady:
print(silnia(0))  # 1
print(silnia(1))  # 1
print(silnia(6))  # 720
```

**Symulacja wykonania dla n=3:**

- silnia(3) → 3 * silnia(2)
- silnia(2) → 2 * silnia(1)
- silnia(1) → 1
- Wracając: silnia(2) = 2 * 1 = 2, silnia(3) = 3 * 2 = 6.

## 4. Wyszukiwanie liniowe (Linear Search)

**Wyszukiwanie liniowe to najprostszy algorytm wyszukiwania elementu w liście (tablicy).** Polega na sprawdzaniu każdego elementu po kolei, od początku do końca, aż znajdzie się poszukiwany element lub dojdzie do końca listy. Nie wymaga, aby lista była posortowana, co jest jego zaletą w porównaniu do wyszukiwania binarnego.

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1                   #


lista = [64, 34, 25, 12, 22, 11, 90]
indeks = linear_search(lista, 25)
print(indeks)

owoce = ["banan", "jabłko", "gruszka", "pomarańcza"]
indeks = linear_search(owoce, "jabłko")
```

```python
print(indeks)

indeks = linear_search(owoce, "wiśnia")
print(indeks)
```